2. Creating a CoAP+DTLS server with ESP-IDF

The following example introduces how to create a CoAP+DTLS server. This example is actually the same as the CoAP example introduced in Section 8.3.4, except that two functions are added to support the DTLS protocol. The function `coap_context_set_psk()` is used to set the PSK encryption key in the DTLS protocol, and can also use certificate (PKI) for the DTLS protocol handshake. The `coap_new_endpoint(ctx, &serv_addr, COAP_PROTO_DTLS)` function indicates that the node supports the DTLS protocol.

Source code

For the complete example code of `coap_context_set_psk()`, please refer to `book-esp32c3-iot-projects/test_case/coap`. For instructions on how to use `PKI`, please see `esp-idf/examples/protocols/coap_server`.

```
1.  static char psk_key[] = "esp32c3_key";
2.  static void esp_create_coaps_server(void)
3.  {
4.      coap_context_t *ctx = NULL;
5.      coap_address_t serv_addr;
6.      coap_resource_t *resource = NULL;
7.      while (1) {
8.          coap_endpoint_t *ep = NULL;
9.          unsigned wait_ms;
10.
11.         //Create a CoAP server socket
12.         coap_address_init(&serv_addr);
13.         serv_addr.addr.sin6.sin6_family = AF_INET6;
14.         serv_addr.addr.sin6.sin6_port = htons(COAP_DEFAULT_PORT);
15.
16.         //Create CoAP ctx
17.         ctx = coap_new_context(NULL);
18.         if (!ctx) {
19.             ESP_LOGE(TAG, "coap_new_context() failed");
20.             continue;
21.         }
22.
23.         //Add PSK encryption key
24.         coap_context_set_psk(ctx, "CoAP",
25.                              (const uint8_t *)psk_key,
26.                              sizeof(psk_key) - 1);
27.
28.         //Set CoAP node
29.         ep = coap_new_endpoint(ctx, &serv_addr, COAP_PROTO_UDP);
30.         if (!ep) {
31.             ESP_LOGE(TAG, "udp: coap_new_endpoint() failed");
32.             goto clean_up;
```

```
        }

        //Add DTLS node and port
        if (coap_dtls_is_supported()) {
            serv_addr.addr.sin6.sin6_port = htons(COAPS_DEFAULT_PORT);
            ep = coap_new_endpoint(ctx, &serv_addr, COAP_PROTO_DTLS);
            if (!ep) {
                ESP_LOGE(TAG, "dtls: coap_new_endpoint() failed");
                goto clean_up;
            } else {
                ESP_LOGI(TAG, "MbedTLS (D)TLS Server Mode not configured");
            }
        }

        //Set CoAP resource URI
        resource = coap_resource_init(coap_make_str_const("light"), 0);
        if (!resource) {
            ESP_LOGE(TAG, "coap_resource_init() failed");
            goto clean_up;
        }

 //Register callback functions for GET and PUT method corresponding to CoAP resource URI
        coap_register_handler(resource, COAP_REQUEST_GET, esp_coap_get);
        coap_register_handler(resource, COAP_REQUEST_PUT, esp_coap_put);

        //Set CoAP GET resource visible
        coap_resource_set_get_observable(resource, 1);

        //Add resource to CoAP ctx
        coap_add_resource(ctx, resource);
        wait_ms = COAP_RESOURCE_CHECK_TIME * 1000;
        while (1) {
            //Wait to receive CoAP data
            int result = coap_run_once(ctx, wait_ms);
            if (result < 0) {
                break;
            } else if (result && (unsigned)result < wait_ms) {
                //Decrease waiting time
                wait_ms -= result;
            } else {
                //Reset waiting time
                wait_ms = COAP_RESOURCE_CHECK_TIME * 1000;
            }
        }
    }
clean_up:
    coap_free_context(ctx);
```

```
80.     coap_cleanup();
81. }
```

8.5 Practice: Local Control in Smart Light Project

The local control component (`esp_local_ctrl`) of ESP-IDF enables you to control Espressif chips via Wi-Fi+HTTPS or Bluetooth LE easily. With this component, you can access application-defined properties, which can be read from or written to through a set of configurable handlers. This section mainly introduces the local control module based on Wi-Fi. Taking the smart light as an example, the local control module can be configured as follows:

- Configure the local device to discover mDNS protocol.
- Configure the local HTTPS server and certificate for data communication.
- Configure the smart lights.

The previous sections have introduced the control of ESP32-C3 via Bluetooth LE. When using Bluetooth LE, the TCP/IP protocol stack is not involved, and local device discovery is not necessary. Bluetooth has its own resource discovery service.

8.5.1 Creating a Wi-Fi-based Local Control Server

The following sample code implements the Wi-Fi-based local control server. The local control is based on HTTP for data communication, and the data is encrypted using the TLS protocol. Additionally, the sample adds the mDNS module for device discovery.

Source code

For the complete example code, please refer to `book-esp32c3-iot-projects/test_case/local_control`.

```
1.  #define PROPERTY_NAME_STATUS "status"
2.  static char light_status[64] = "{\"status\": true}";
3.
4.  //Property type definition, used with scripts
5.  enum property_types {
6.      PROP_TYPE_TIMESTAMP = 0,
7.      PROP_TYPE_INT32,
8.      PROP_TYPE_BOOLEAN,
9.      PROP_TYPE_STRING,
10. };
11.
12. //Get attribute value
13. esp_err_t get_property_values(size_t props_count,
14.                               const esp_local_ctrl_prop_t props[],
15.                               esp_local_ctrl_prop_val_t prop_values[],
16.                               void *usr_ctx)
17. {
```

```
18.     int i = 0;
19.     for (i = 0; i < props_count; i ++) {
20.         ESP_LOGI(TAG, "Reading property : %s", props[i].name);
21.         if (!strncmp(PROPERTY_NAME_STATUS,
22.                     props[i].name,
23.                     strlen(props[i].name))) {
24.             prop_values[i].size = strlen(light_status);
25.             prop_values[i].data = &light_status;//prop_values[i].data is
26.                                 //just a pointer, and cannot be assigned.
27.             break;
28.         }
29.     }
30.     if (i == props_count) {
31.         ESP_LOGE(TAG, "Not found property %s", props[i].name);
32.         return ESP_FAIL;
33.     }
34.     return ESP_OK;
35. }
36.
37. //Set property value
38. esp_err_t set_property_values(size_t props_count,
39.                               const esp_local_ctrl_prop_t props[],
40.                               const esp_local_ctrl_prop_val_t prop_values[],
41.                               void *usr_ctx)
42. {
43.     int i = 0;
44.     for (i = 0; i < props_count; i ++) {
45.         ESP_LOGI(TAG, "Setting property : %s", props[i].name);
46.         if (!strncmp(PROPERTY_NAME_STATUS,
47.                     props[i].name,
48.                     strlen(props[i].name))) {
49.             memset(light_status, 0, sizeof(light_status));
50.             strncpy(light_status,
51.                     (const char *)prop_values[i].data,
52.                     prop_values[i].size);
53.             if (strstr(light_status, "true")) {
54.                 app_driver_set_state(true);  //Turn on the smart light
55.             } else {
56.                 app_driver_set_state(false); //Turn off the smart light
57.             }
58.             break;
59.         }
60.     }
61.     if (i == props_count) {
62.         ESP_LOGE(TAG, "Not found property %s", props[i].name);
63.         return ESP_FAIL;
64.     }
```

```
65.     return ESP_OK;
66. }
67. #define SERVICE_NAME "my_esp_ctrl_device"
68. void esp_local_ctrl_service_start(void)
69. {
70.     //Initialise the HTTPS server-side configuration
71.     httpd_ssl_config_t https_conf = HTTPD_SSL_CONFIG_DEFAULT();
72.     //Load the server certificate
73.     extern const unsigned char cacert_pem_start[] asm("_binary_cacert_pem_ start");
74.     extern const unsigned char cacert_pem_end[]   asm("_binary_cacert_pem_end");
75.     https_conf.cacert_pem = cacert_pem_start;
76.     https_conf.cacert_len = cacert_pem_end - cacert_pem_start;
77.     //Load server-side private key
78.     extern const unsigned char prvtkey_pem_start[] asm("_binary_prvtkey_pem_ start");
79.     extern const unsigned char prvtkey_pem_end[] asm("_binary_prvtkey_pem_end");
80.     https_conf.prvtkey_pem = prvtkey_pem_start;
81.     https_conf.prvtkey_len = prvtkey_pem_end - prvtkey_pem_start;
82.     esp_local_ctrl_config_t config = {
83.         .transport = ESP_LOCAL_CTRL_TRANSPORT_HTTPD,
84.         .transport_config = {
85.             .httpd = &https_conf
86.         },
87.         .proto_sec = {
88.             .version = PROTOCOM_SEC0,
89.             .custom_handle = NULL,
90.             .pop = NULL,
91.         },
92.         .handlers = {
93.
94.             //User-defined processing function
95.             .get_prop_values = get_property_values,
96.             .set_prop_values = set_property_values,
97.             .usr_ctx           = NULL,
98.             .usr_ctx_free_fn = NULL
99.         },
100.
101.        //Set the maximum number of attributes
102.        .max_properties = 10
103.    };
104.
105.    //Initialise local discovery
106.    mdns_init();
107.    mdns_hostname_set(SERVICE_NAME);
108.
109.    //Start the local control service
110.    ESP_ERROR_CHECK(esp_local_ctrl_start(&config));
111.    ESP_LOGI(TAG, "esp_local_ctrl service started with name : %s", SERVICE_NAME);
```

```
112.    esp_local_ctrl_prop_t status = {
113.        .name        = PROPERTY_NAME_STATUS,
114.        .type        = PROP_TYPE_STRING,
115.        .size        = 0,
116.        .flags       = 0,
117.        .ctx         = NULL,
118.        .ctx_free_fn = NULL
119.    };
120.    //Add attribute value
121.    ESP_ERROR_CHECK(esp_local_ctrl_add_property(&status));
122.}
```

The above sample code implements the discovery of a device (domain name: `my_esp_ctrl_device.local`) through the local discovery protocol (mDNS), establishes an HTTPS local control connection, and allows the client to set and query attribute values via a registered endpoint.

Users can enable the transmission security protection for local control via the following options:

- PROTOCOM_SEC0: specifies the end-to-end encryption algorithm used.
- PROTOCOM_SEC1: specifies that data is exchanged as plain text.
- PROTOCOM_SEC_CUSTOM: customises security requirements.

Each attribute must have a unique name (a string), type (e.g., int, bool, or string), flag (e.g., read-only, or readable and writable), and size. If the property value is expected to be of variable length (e.g., if the property value is a string or byte stream), the `size` should be kept at 0. For fixed-length property value data types, such as int, float, etc., setting the size field to the correct value helps `esp_local_ctrl` perform internal checks on parameters received via write requests.

You can process it by matching `props[i].name` with the corresponding property name, and further checking the `flag` and `type` of the property to determine if the property satisfies the corresponding `flag` and `type` requirements.

The default endpoints are shown in Table 8.5.

Table 8.5. Default endpoints

Endpoint name (BLE + GATT Server)	URI (HTTPS Server + mDNS)	Description
esp_local_ctrl/version	https://my_esp_ctrl_device.local/esp_local_ctrl/version	For retrieving version strings
esp_local_ctrl/control	https://my_esp_ctrl_device.local/esp_local_ctrl/control	For sending/receiving control messages

8.5.2 Verifying Local Control Functionality using Scripts

After introducing how to create a local control module, this section will further introduce how to use scripts for verification. Here, we use the official example `esp_local_ctrl` as an example for verification.

1. Create a certificate for TLS handshake between the client and the server.

a. Generate a `rootCA` that will be used to sign the server-side certificate, and the client will use it to verify the server-side certificate during the SSL handshake. A passphrase needs to be set to encrypt the generated `rootkey.pem`.

```
$ openssl req -new -x509 -subj "/CN=root" -days 3650 -sha256 -out rootCA.pem
-keyout rootkey.pem
```

b. Generate a certificate signing request and its private key `prvtkey.pem` for the server.

```
$ openssl req -newkey rsa:2048 -nodes -keyout prvtkey.pem -days 3650 -out
server.csr -subj "/CN=my_esp_ctrl_device.local"
```

c. Use the previously generated `rootCA` to process the server-side certificate signing request and generate the signing certificate `cacert.pem`. The passphrase set earlier for the encrypted `rootkey.pem` must be entered in this step.

```
$ openssl x509 -req -in server.csr -CA rootCA.pem -CAkey rootkey.pem
-CAcreateserial -out cacert.pem -days 500 -sha256
```

Among the generated certificates, `cacert.pem` and `prvtkey.pem` are compiled into the server, and `rootkey.pem` is suitable for client-side scripts for server-side verification. The directory of the certificate can be set in the script `esp_local_ctrl.py`.

```
1.  def get_transport(sel_transport, service_name, check_hostname):
2.  ...
3.      example_path = os.environ['IDF_PATH'] + '/examples/protocols/esp_local_ctrl'
4.      cert_path = example_path + '/main/certs/rootCA.pem'
5.  ...
```

2. Use the following command to connect to the local control server via script. If `sec_ver` is 0, it means that `PROTOCOM_SEC0` is set on the server.

```
$ python esp_local_ctrl.py --sec_ver 0
```

The script will automatically get the property value, i.e.:

```
Connecting to my_esp_ctrl_device.local

==== Starting Session ====
==== Session Established ====

==== Available Properties ====
S.N. Name              Type       Flags            Value

[1] status           STRING                     {"status": true}
```

3. According to the script prompt, enter the attribute number “1”, and set the attribute value to `{"status": false}`. Then the script will automatically start querying and find that the property value has been changed.

```
Select properties to set (0 to re-read, 'q' to quit) : 1
Enter value to set for property (status) : {"status": false}
==== Available Properties ====
S.N. Name            Type       Flags            Value
[1] status           STRING                      {"status": false}

Select properties to set (0 to re-read, 'q' to quit) :
```

8.5.3 Creating a Bluetooth-based Local Control Server

The following sample code creates a Bluetooth-based local control server, which can be used to transmit data. You can refer to the `gatt_server` sample. The following example implements a Bluetooth server. You can compile and flash this example, then use a Bluetooth debugging tool on your smartphone to scan and connect to a Bluetooth device named `ESP32C3-LIGHT`. Once connected, you can access the Bluetooth services provided by the device.

Source code

For the complete code of the `gatt_server` example, please refer to `book-esp32c3-iot-projects/test_case/gatt_server`.

This example provides two Bluetooth services: one for getting the device status (`UUID: FF01`) and another for setting the device status (`UUID: EE01`), as shown in Figure 8.19.

The running log is as follows.

```
I (387) GATTS_DEMO: NVS Flash initialization
I (387) GATTS_DEMO: Application driver initialization
I (397) gpio: GPIO[9]| InputEn: 1| OutputEn: 0| OpenDrain: 0| Pullup: 1|
Pulldown: 0| Intr:0
W (437) BTDM_INIT: esp_bt_controller_mem_release not implemented, return OK
I (437) BTDM_INIT: BT controller compile version [501d88d]
I (437) coexist: coexist rom version 9387209
I (437) phy_init: phy_version 500,985899c,Apr 19 2021,16:05:08
I (617) system_api: Base MAC address is not set
I (617) system_api: read default base MAC address from EFUSE
I (617) BTDM_INIT: Bluetooth MAC: 68:ab:bc:a7:d8:d5
I (637) GATTS_DEMO: REGISTER_APP_EVT, status 0, app_id 0
I (647) GATTS_DEMO: CREATE_SERVICE_EVT, status 0,  service_handle 40
I (647) GATTS_DEMO: SERVICE_START_EVT, status 0, service_handle 40
I (647) GATTS_DEMO: ADD_CHAR_EVT, status 0,  attr_handle 42, service_handle 40
I (657) GATTS_DEMO: ADD_DESCR_EVT, status 0, attr_handle 43, service_handle 40
I (667) GATTS_DEMO: REGISTER_APP_EVT, status 0, app_id 1
```

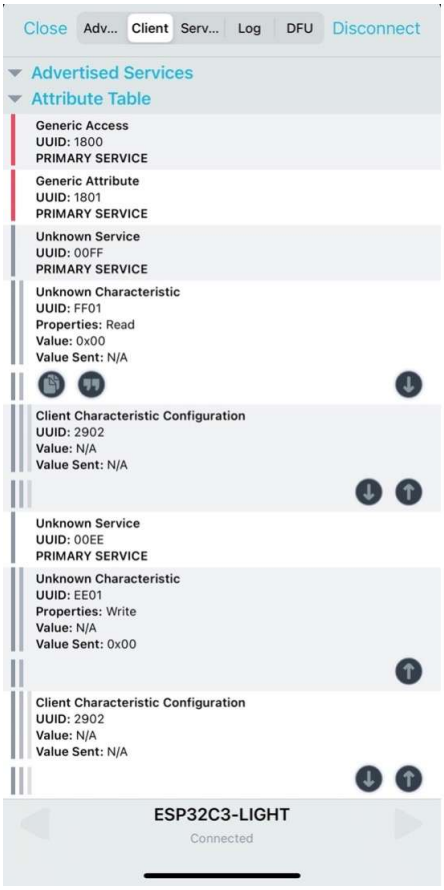


Figure 8.19. Bluetooth services in the example

```
I (677) GATTS_DEMO: CREATE_SERVICE_EVT, status 0,  service_handle 44
I (677) GATTS_DEMO: SERVICE_START_EVT, status 0, service_handle 44
I (687) GATTS_DEMO: ADD_CHAR_EVT, status 0,  attr_handle 46, service_handle 44
I (697) GATTS_DEMO: ADD_DESCR_EVT, status 0, attr_handle 47, service_handle 44
I (6687) GATTS_DEMO: ESP_GATTS_CONNECT_EVT, conn_id 0, remote 4a:13:d8:ca:b3:cf:
I (6687) GATTS_DEMO: CONNECT_EVT, conn_id 0, remote 4a:13:d8:ca:b3:cf:
I (6987) GATTS_DEMO: ESP_GATTS_MTU_EVT, MTU 500
I (6987) GATTS_DEMO: ESP_GATTS_MTU_EVT, MTU 500
I (7347) GATTS_DEMO: update connection params status = 0, min_int = 16, max_int
= 32,conn_int = 24,latency = 0, timeout = 400
I (15117) GATTS_DEMO: GATT_READ_EVT, conn_id 0, trans_id 3, handle 42
I (23037) GATTS_DEMO: GATT_WRITE_EVT, conn_id 0, trans_id 4, handle 46
I (23037) GATTS_DEMO: GATT_WRITE_EVT, value len 1, value :
I (23037) GATTS_DEMO: 00
I (23037) app_driver: Light OFF
I (30987) GATTS_DEMO: GATT_WRITE_EVT, conn_id 0, trans_id 5, handle 46
I (30987) GATTS_DEMO: GATT_WRITE_EVT, value len 1, value :
I (30987) GATTS_DEMO: 01
I (30987) app_driver: Light ON
```

8.6 Summary

In this chapter, we first presented an overview of the framework model, applicable conditions, and application scenarios of local control. We also compared it to remote control, enabling you to assess the suitability of local control functionality based on your unique project requirements. Local discovery plays a pivotal role in local control as it governs the ability of smartphones to search for devices within the LAN. This allows smartphones to retrieve device characteristics and facilitates subsequent control of the identified devices. Therefore, we also delved into the operational mode of the local discovery protocols, specifically in terms of the principle layer. We also conducted a comparative analysis of the characteristics of the two modes: broadcast and multicast, shedding light on their similarities and differences. The most commonly used local discovery protocol is mDNS, which provides you with the flexibility to implement local discovery functionality in your IoT projects. You can also leverage the resource discovery technology inherent in the Bluetooth protocol directly when utilising Bluetooth for control purposes.

Then we introduced the most critical data communication protocols and corresponding data encryption algorithms in local control. The fundamental protocols for data communication in local control are TCP and UDP . While you can directly utilise these protocols for local control, it is generally not recommended due to certain limitations. TCP and UDP are classified as transport layer protocols and do not inherently carry application format data, in contrast to protocols like HTTP and CoAP, which incorporate an application layer on top of the transport layer. Furthermore, it's important to note that transport layer protocols such as TCP and UDP do not support direct encryption of data using protocols like TLS or DTLS. Therefore, relying solely on the transport layer protocol for data transmission may not guarantee the security of the transmitted data. Hence, it is recommended that you utilise protocols such as HTTP with TLS or CoAP with DTLS for data communication in local control scenarios.

Finally, we introduced how to implement the complete local control function based on the `esp_local_ctrl` component in ESP-IDF. `esp_local_ctrl` supports local control based on Wi-Fi and Bluetooth, with data communication protocols that include HTTPS for Wi-Fi and Bluetooth protocols for Bluetooth. You can get started with local control development with `esp_local_ctrl` component. Additionally, the `esp_local_ctrl` component does not support local network device discovery functionality. You need to implement device discovery functionality using the mDNS module. The `esp_local_ctrl` component is widely used in ESP-IDF, and you can find provisioning and local communication features in the `wifi_provisioning` component for network configuration and local communication.

Of course, there are various protocols and implementation methods for local control. If the `esp_local_ctrl` component does not meet your requirements, you can follow the example codes in sections 8.2, 8.3, and 8.4 to build your own local control framework.

Chapter 9 Cloud Control

After reading the local control introduced in Chapter 8, you should know how to design the local control function for your IoT projects. However, this function is far from enough, because a complete IoT project aims to connect all things and local control has a geographical limitation: the smartphone must be in the same local area network (LAN) as the controlled device. If you want to remotely control the IoT devices at home through your smartphone, you will need the remote control function.

This chapter mainly introduces how to remotely control devices based on ESP32-C3. The purpose is to help you understand what remote control and its process are, what protocol is involved, how to build an MQTT server locally to simulate the cloud server, and how to build a product model through ESP RainMaker for remote control of a device.

9.1 Introduction to Remote Control

What is remote control? As the name suggests, remote control refers to the behaviour of one device (such as smartphones, computers, or other network devices) controlling another device through a wide area network (WAN). It is not restricted by region. For example, you can control smart lights at home through your smartphone in the workplace. In general, both the remotely-controlling device and the remotely-controlled device need to be connected to the cloud server, and the commands sent by the controlling device are transmitted to the controlled device over the cloud server.

Similar to local control (covered in Chapter 8), remote control is also a way of data communication, but it is over WAN other than LAN. In local control, the server can be the controlled device itself, or a host in LAN; the controlling device (such as mobile phones or computers) must be in the same LAN as the server, which is a limitation. In remote control, the server is generally a cloud server (several large-scale cloud server providers are Alibaba Cloud, Amazon Cloud, Tencent Cloud, etc.), the controlling device and the controlled device need to be connected to the cloud server, and the data forwarding and storage are handled by the cloud server.

The advantage of remote control is that the control is flexible and can break through the limitation of space. However, compared with local control, it requires cloud services and

network traffic, thus more costly. Moreover, it usually has higher latency, resulting in a greater risk of leaking data.

As the implementation principle and components of ESP RainMaker covered in Section 3.2 indicate, in remote control, both the controlling device (smartphone) and the controlled device (such as ESP32-C3) are directly connected to the cloud server, which facilitates the transfer of data between the devices. As a result, it is essential to have a thorough understanding of how these devices communicate with the cloud server.

Remote control costs more than local control as it requires cloud servers, but it is more convenient to remotely view the operating status of the controlled device. Both have their advantages and disadvantages. At present, most of the IoT devices on the market can be connected to various clouds. For instance, the products of Xiaomi, Alibaba, and JD are connected to their own cloud platform. The user only needs to download the corresponding app and perform the provision and binding to view and control their IoT devices.

If your smartphone and the controlled device are on the same LAN, local control is a better option. Otherwise, remote control has to be used. Local control has its own use scenarios and advantages. The advantages of both should be fully utilized to develop the most suitable IoT control technology.

9.2 Cloud Data Communication Protocols

Section 9.1 described what remote control was. As the topological structure of remote control shows, the smartphone and the controlled device are not directly connected. They are connected to the cloud server, and the data sent by both are forwarded by the cloud. Then, what is the protocol for connecting the device to the cloud? What is the protocol for data communication? Only by figuring out these protocols can you have a basic understanding of remote control.

At present, common protocols for connecting devices to the cloud are the MQTT protocol and HTTP protocol. This chapter will only cover the former as the latter has been introduced in Chapter 8.

9.2.1 MQTT Introduction

MQTT (Message Queue Telemetry Transport) is a server-client publish-subscribe messaging transmission protocol. It is open, simple, lightweight, standardized, and easy to implement. These characteristics make it a standard IoT transmission protocol that is ideal for resource-constrained devices. The protocol was released by IBM in 1999. At present, it has been developed to v5.x, and ESP-IDF supports v3.1.1. The two versions have significant differences and are not compatible with each other. Most cloud platforms currently still rely on the older v3.x version. Therefore, in this chapter, we will be focusing on MQTT v3.x.

The MQTT protocol runs over the TCP protocol. It has the following features:

- The publish/subscribe pattern which supports one-to-many message distribution and decoupling of applications.
- A messaging transport that is agnostic to the content of the payload.
- Three qualities of service (QoS) for message delivery.
- Small transport overhead and protocol exchanges minimised to reduce network traffic.
- Will messages to notify interested parties when an abnormal disconnection occurs.

9.2.2 MQTT Principles

The MQTT protocol is based on client-server communication. It defines three roles: publisher, broker, and subscriber. The publisher and the subscriber serve as the Client, which can both publish and subscribe messages. The broker acts as the Server. Figure 9.1 shows the architecture of the protocol.

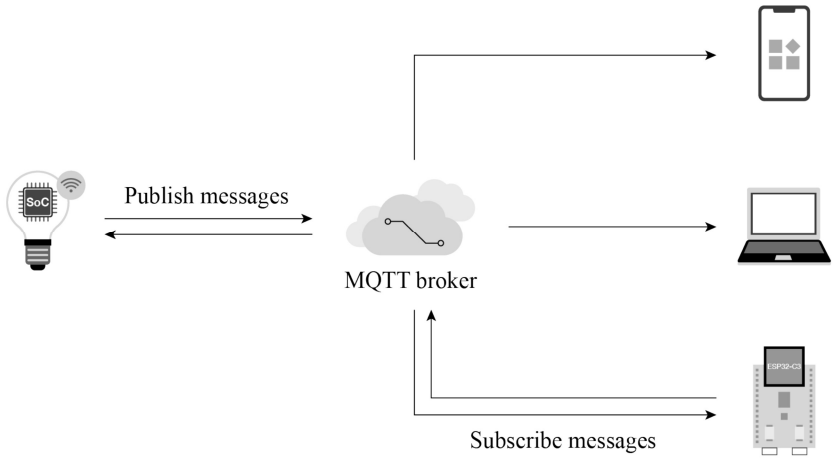


Figure 9.1. Architecture of MQTT protocol

(1) **Client**: a device running MQTT applications, such as smartphones and controlled devices. It can work as a publisher or subscriber. A Client always connects to the Server over the network. It can:

- Publish application messages that other Clients might be interested in.
- Subscribe to request application messages that it is interested in receiving.
- Unsubscribe to remove a request for application messages.
- Disconnect from the Server.

(2) **Server**: a broker that acts as an intermediary between Clients publishing application messages and Clients requesting to subscribe to them, such as cloud platforms and cloud servers. It can:

- Accept network connections from Clients.
- Accept application messages published by Clients.
- Process subscribe and unsubscribe requests from Clients.
- Forward application messages that match Client subscriptions.

(3) **Subscription**: comprising a Topic Filter and a maximum QoS. It is associated with a single Session, while a Session may contain multiple Subscriptions. Each Subscription within a session has a different Topic Filter.

(4) **Topic**: the label attached to an application message. It is matched against the Subscriptions known to the Server. The Server sends a copy of the application message to each Client that has a matching Subscription.

(5) **Topic Filter**: an expression contained in a Subscription, indicating an interest in one or more topics. A Topic Filter can include wildcard characters to represent single or multiple characters.

(6) **Session**: a stateful interaction between a Client and a Server from the start to the end of a connection. Some Sessions last only as long as the Network Connection.

(7) **Publish/Subscribe**: the core of the MQTT protocol. It allows communication between subscribers and publishers without knowledge of each other's IP address or port number. Direct connection is not even necessary, and subscribers and publishers can operate without knowing each other's existence. Message exchanges between them are done by the broker, which filters all the published messages and then distributes them to the matching subscribers.

Both subscribers and publishers are concerned about the topic of the message. For example, a smartphone wants to check the status of a smart light A. In this case, the smartphone can act as a subscriber to subscribe to the message with the topic `A/light_state` from the broker. Smart light A can act as a publisher. When its state changes, it publishes a status message with the same topic to the broker. Then, the broker filters subscribers who have subscribed to the topic and publishes the status message to them. In this way, the smartphone can query the status of smart light A.

9.2.3 MQTT Message Format

As defined by the MQTT protocol, an MQTT control packet consists of three parts: fixed header, variable header, and payload.

(1) **Fixed header**, present in all MQTT control packets.

As shown in Figure 9.2, the packet type takes 4 bits.

There are 14 types of control packets in total, as listed in Table 9.1.

Bit	7	6	5	4	3	2	1	0
Byte 1	Type of the control packet				Flag specific to the type of the control packet			
Byte 2	Remaining length							

Figure 9.2. Fixed header of MQTT control packets

Table 9.1. Types of MQTT control packets

Name	Value	Direction of Flow	Description
Reserved	0	Forbidden	Reserved
CONNECT	1	Server ⇐ Client	Client request to connect to Server
CONNACK	2	Server ⇒ Client	Connection acknowledgement
PUBLISH	3	Server ⇔ Client	Publish message
PUBACK	4	Server ⇔ Client	Publish of QoS 1 message acknowledged
PUBREC	5	Server ⇔ Client	Publish received (assured delivery part 1)
PUBREL	6	Server ⇔ Client	Publish release (assured delivery part 2)
PUBCOMP	7	Server ⇔ Client	Publish of QoS 2 message complete (assured delivery part 3)
SUBSCRIBE	8	Server ⇐ Client	Subscribe request of Client
SUBACK	9	Server ⇒ Client	Subscribe acknowledgement
UNSUBSCRIBE	10	Server ⇐ Client	Unsubscribe request
UNSUBACK	11	Server ⇒ Client	Unsubscribe acknowledgement
PINGREQ	12	Server ⇐ Client	PING request
PINGRESP	13	Server ⇒ Client	PING response
DISCONNECT	14	Server ⇐ Client	Client is disconnecting

There are three MQTT QoS levels: QoS 0, QoS 1, and QoS 2.

a. **QoS 0**: delivered once at most.

   The transmission of messages is completely dependent on the underlying TCP/IP network. As the MQTT protocol does not define response and retry, the message will either reach the server only once or not at all. The flow of MQTT QoS 0 is shown in Figure 9.3.

b. **QoS 1**: delivered once at least.

   The acknowledgement of message receipt is provided by the PUBACK message. If the communication link or the sending device is abnormal, or the message is not received

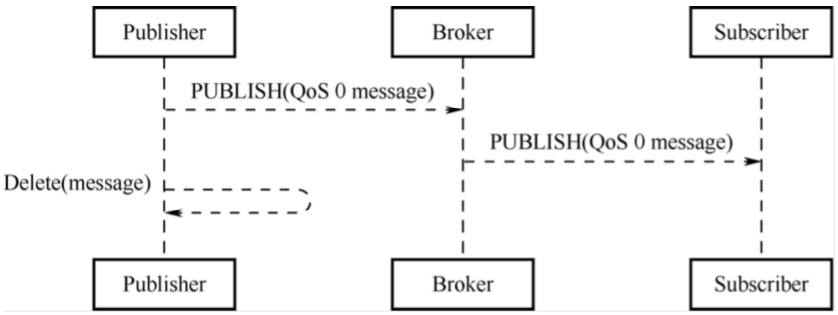


Figure 9.3. Flow of MQTT QoS 0

within the specified time, the sender will redeliver the message, and in the fixed header of the MQTT Control packet set the duplicate flag (DUP). The flow of MQTT QoS 1 is shown in Figure 9.4.

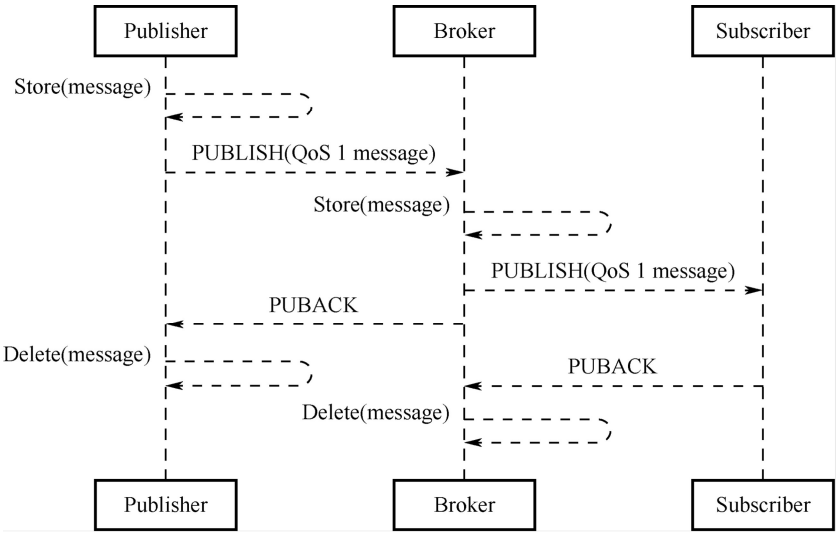


Figure 9.4. Flow of MQTT QoS 1

c. **QoS 2**: delivered only once.

   This is the highest quality of service where message loss and duplication are unacceptable and increased overhead is incurred. The flow of MQTT QoS 2 is shown in Figure 9.5.

Bits [3–0] of the fixed header contain flags specific to each type of control packets. Except the PUBLISH type, the flag bits of other types are taken by the system. For types without flags, the bits are reserved. If invalid flags are received, the receiver must close the network connection. The bits [3–0] in byte 1 of the PUBLISH packet header are as follows:

a. **DUP (bit3)**: duplicate delivery.

   "0" means that this is the first time that the Client or Server requests to send a PUBLISH

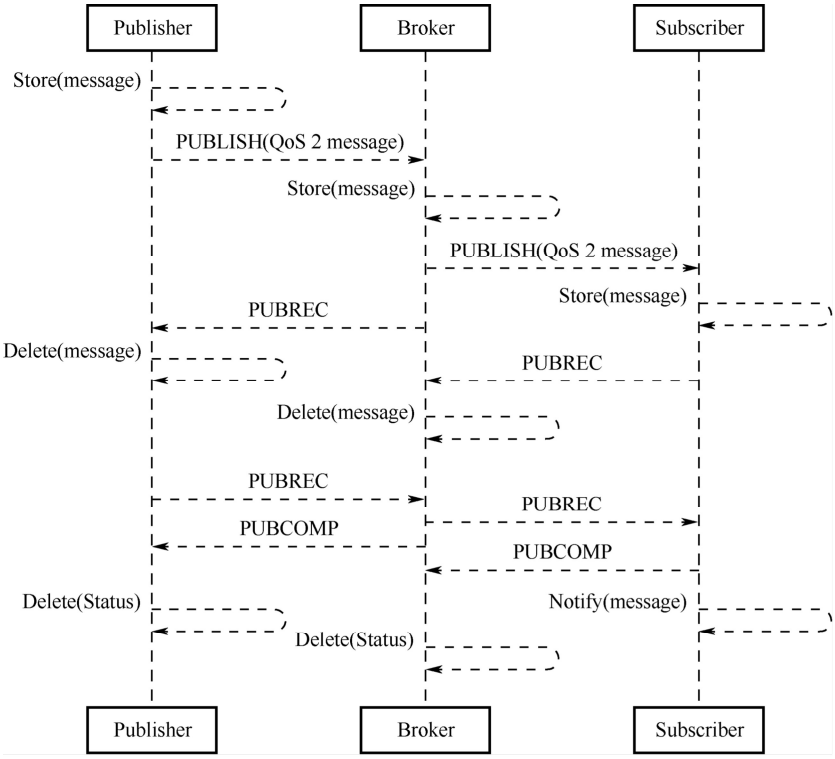


Figure 9.5. Flow of MQTT QoS 2

message. “1” indicates that this may be a duplicate delivery of an earlier message. The DUP flag of QoS 0 messages must be set to 0.

b. **QoS (bits [2–1])**: determining the number of message delivery.

Table 9.2 shows how to represent QoS values in bits [2–1].

Table 9.2. Representation of QoS values in bits [2–1]

QoS Value	Bit2	Bit1	Description
0	0	0	Delivered once at most
1	0	1	Delivered once at least
2	1	0	Delivered only once
—	1	1	Reserved

c. **RETAIN (bit0)**: determining the need for message retaining.

If this flag is set to 1 in a PUBLISH message sent by the Client to the Server, the Server must store this message and its QoS to distribute them later to subscribers with a matching topic. Each Client subscribing to a topic pattern that matches the topic of the retained

message receives the retained message immediately after they subscribe. The RETAIN flag is usually used for will messages. For example, after a device is unexpectedly disconnected, the broker will send the will message to the smartphone, where the device will be displayed as offline.

The second and subsequent bytes indicate the remaining length, indicating the number of remaining bytes within the current packet, including data in the variable header and the payload. The remaining length is encoded using a variable-length encoding scheme that uses a single byte for values up to 127. For larger values, the least significant seven bits of each byte encode the data, and the most significant bit is used to indicate whether there are following bytes in the representation. Thus, each byte encodes 128 values and a "continuation bit". The maximum number of bytes in the remaining length field is 4B. The number of remaining length bytes is shown in Table 9.3.

Table 9.3. Number of remaining length bytes

Byte	Minimum Value	Maximum Value
1	0 (0x00)	127 (0x7F)
2	128 (0x80, 0x01)	16383 (0xFF, 0x7F)
3	16384 (0x80, 0x80, 0x01)	2097151 (0xFF, 0xFF, 0x7F)
4	2097152 (0x80, 0x80, 0x80, 0x01)	268435455 (0xFF, 0xFF, 0xFF, 0x7F)

(2) **Variable header.**

Some types of MQTT Control Packets contain a variable header component. It resides between the fixed header and the payload. The content of the variable header varies depending on the packet type. The Packet Identifier field of the variable header is present in multiple types of packets, such as PUBLISH (when QoS¿0), PUBACK, PUBREC, PUBREL, PUBCOMP, SUBSCRIBE, SUBACK, UNSUBSCRIBE, and UNSUBACK.

(3) **Payload**: the third part of an MQTT control packet, containing the message content.

It exists in five types of packets, CONNECT, SUBSCRIBE, SUBACK, UNSUBSCRIBE, and PUBLISH.

a. CONNECT: Client ID, topic, message, user name, and password.
b. SUBSCRIBE: a series of topics to subscribe to and the QoS.
c. SUBACK: the server's acknowledgment and reply to the topic (that the Client requests to subscribe) and the QoS.
d. UNSUBSCRIBE: the topics to unsubscribe.
e. PUBLISH: the to-be-published application message, which can be zero-length.

9.2.4 Protocol Comparison

Chapter 8 introduced protocols such as TCP, HTTP, UDP, and CoAP, which can facilitate local control. In addition, they can also be used for remote control.

MQTT vs. TCP

MQTT is an application protocol based on the TCP protocol. Both can be used for remote data communication. For socket programming, TCP requires users to develop their own application protocols, which have limited usability in the current environment of IoT interconnection. On the other hand, MQTT is a standardized lightweight protocol for IoT and is widely used by most cloud servers, such as Alibaba Cloud and Amazon Cloud, making it advantageous for product integration.

MQTT vs. HTTP

Both are client-server application protocols based on TCP. However, compared to MQTT, HTTP has a much larger overhead in message size, and it is generally difficult for an HTTP server to initiate data push to clients, which may not meet the requirements of remote control in IoT. In cases where only one-way transmission from clients to the server is needed, HTTP protocol can be used.

MQTT vs. CoAP

Similar to HTTP, CoAP adopts the REST model where the server creates resources in URI format and clients access these resources using GET, PUT, POST, and DELETE methods. Besides, CoAP also has a similar protocol style to HTTP, but it requires fewer device resources and network overheads, making it suitable for IoT. However, CoAP may not be a good choice for remote control. If smartphones send control commands for remote control, the architecture may require CoAP + Web + Database + App. When CoAP protocol is used, control commands must pass through the Database before reaching the device, because CoAP is connectionless. When smartphones send control commands, the server will first store the control commands in the Database, and the device will request the server via GET method to check if there are any control commands, and then decide whether to operate. On the other hand, MQTT is connection-oriented, and the server will forward the control commands from smartphones to all subscribed devices without storing them. Only MQTT client + MQTT server + App is needed to implement remote control, making MQTT more advantageous in terms of deployment.

9.2.5 Setting Up MQTT Broker on Linux and Windows

Some commonly used MQTT brokers are Mosquitto, EMQTT, and HiveMQ. HiveMQ is not open-source and has a fee, so it may not be suitable for local testing. EMQTT has powerful features, such as viewing data traffic on a web interface, and can be used on most cloud servers, with both free and paid custom versions available.

This section focuses on how to use Mosquitto to set up an MQTT broker on Windows or Linux. Mosquitto is an open-source (EPL/EDL licensed) message broker that implements MQTT protocol versions 5.0, 3.1.1, and 3.1. It is considered a lightweight open-source software. The Mosquitto project provides a C language library for implementing MQTT clients and popular command-line MQTT clients `mosquitto_pub` and `mosquitto_sub`. Besides, Mosquitto can also be used as an MQTT broker. For more information, please refer to its official website.

1. Setting up MQTT broker on Linux

All terminal commands in this section must be run in the user role. The `$` symbol represents the command prompt.

(1) Download **mosquitto-2.0.12.tar.gz** from https://mosquitto.org/files/source.

(2) Extract Mosquitto.

```
$ tar -zxvf mosquitto-2.0.12.tar.gz
```

Check if the installation is successful using `mosquitto --help`.

```
$ cd mosquitto-2.0.12/src
$ mosquitto --help
mosquitto version 2.0.12
mosquitto is an MQTT v5.0/v3.1.1/v3.1 broker.
    Usage: mosquitto [-c config_file] [-d] [-h] [-p port]
        -c : specify the broker config file.
        -d : put the broker into the background after starting.
        -h : display this help.
        -p : start the broker listening on the specified port.
          Not recommended in conjunction with the -c option.
        -v : verbose mode - enable all logging types. This overrides
          any logging options given in the config file.
        See https://mosquitto.org/ for more information.
```

(3) Start MQTT broker and test in the MQTT client.

a. Start MQTT.

```
$ mosquitto
```

b. Use `mosquitto_sub` to subscribe to `topic`.

```
$ mosquitto_sub -t 'test/topic' -v
```

c. Open a new terminal and use `mosquitto_pub` to publish data.

```
$ mosquitto_pub -t 'test/topic' -m 'hello world'
```

d. In the original terminal where the `topic` was subscribed, view the received data.

```
$ mosquitto_sub -t 'test/topic' -v
test/topic hello world
```

2. Setting up MQTT broker on Windows

(1) Download the 32-bit or 64-bit MQTT installation package based on your computer's architecture. Double-click to install it.

(2) Open a command prompt window, navigate to the directory where Mosquitto is installed, and start the Mosquitto broker.

```
cd C:\Program Files\mosquitto\
```

(3) Use `mosquitto_sub.exe` to subscribe to `topic`.

```
C:\Program Files\mosquitto>mosquitto_sub.exe -t 'test/topic' -v
```

(4) Use `mosquitto_pub.exe` to publish data.

```
C:\Program Files\mosquitto>mosquitto_pub.exe -t 'test/topic' -m 'hello_world'
```

9.2.6 Setting Up MQTT Client Based on ESP-IDF

The component used in ESP-IDF to implement MQTT client is ESP-MQTT, which has the following features:

- Support for MQTT, MQTT over TLS, MQTT over WebSocket, and MQTT over WebSocket, and TLS
- Easy to set up with URI
- Multiple clients in one application
- Support for subscribing, publishing, authentication, last will messages, keep-alive pings, and QoS messages

The following code is based on ESP-IDF and creates a connection to a local MQTT broker. For complete code, please go to ESP-IDF project on GitHub and navigate to the directory `esp-idf/examples/protocols/mqtt/tcp`.

```
1.  //MQTT event handler function
2.  static void mqtt_event_handler(void *handler_args, esp_event_base_t base,
3.                                 int32_t event_id, void *event_data)
4.  {
5.      esp_mqtt_event_handle_t event = event_data;
6.      esp_mqtt_client_handle_t client = event->client;
7.      int msg_id;
8.      switch ((esp_mqtt_event_id_t)event_id) {
9.          case MQTT_EVENT_CONNECTED:
10.         ESP_LOGI(TAG, "MQTT_EVENT_CONNECTED");
11.         //Subscribe to topic /topic/test
12.         msg_id = esp_mqtt_client_subscribe(client, "/topic/test", 0);
13.         ESP_LOGI(TAG, "sent subscribe successful, msg_id=%d", msg_id);
14.         break;
15.         case MQTT_EVENT_DISCONNECTED:
16.         ESP_LOGI(TAG, "MQTT_EVENT_DISCONNECTED");
```

```
17.         break;
18.         case MQTT_EVENT_SUBSCRIBED:
19.         ESP_LOGI(TAG, "MQTT_EVENT_SUBSCRIBED, msg_id=%d", event->msg_id);
20.         break;
21.         case MQTT_EVENT_UNSUBSCRIBED:
22.         ESP_LOGI(TAG, "MQTT_EVENT_UNSUBSCRIBED, msg_id=%d", event->msg_id);
23.         break;
24.         case MQTT_EVENT_PUBLISHED:
25.         ESP_LOGI(TAG, "MQTT_EVENT_PUBLISHED, msg_id=%d", event->msg_id);
26.         break;
27.         case MQTT_EVENT_DATA:
28.         ESP_LOGI(TAG, "MQTT_EVENT_DATA");
29.         ESP_LOGI(TAG, "TOPIC=%.*s\r\n", event->topic_len, event->topic);
30.         ESP_LOGI(TAG, "DATA=%.*s\r\n", event->data_len, event->data);
31.         break;
32.         case MQTT_EVENT_ERROR:
33.         ESP_LOGI(TAG, "MQTT_EVENT_ERROR");
34.         break;
35.         default:
36.         ESP_LOGI(TAG, "Other event id:%d", event->event_id);
37.         break;
38.     }
39. }
40.
41. #define CONFIG_BROKER_URL "mqtt://192.168.3.4/"
42.
43. static void esp_mqtt_start(void)
44. {
45.     //Configure MQTT URI
46.     esp_mqtt_client_config_t mqtt_cfg = {
47.         .uri = CONFIG_BROKER_URL,
48.     };
49.
50.     //Initialise MQTT client
51.     esp_mqtt_client_handle_t client = esp_mqtt_client_init(&mqtt_cfg);
52.
53.     //Register event handler function
54.     esp_mqtt_client_register_event(client, ESP_EVENT_ANY_ID, mqtt_event_handler, NULL);
55.
56.     //Start MQTT client
57.     esp_mqtt_client_start(client);
58. }
```

The client device connects to the MQTT broker and subscribes to the topic `/topic/test`. After another MQTT client publishes the message `hello world` to the topic `/topic/test`, the following log will show up in the device:

```
I (2598) wifi station: MQTT_EVENT_CONNECTED
```

```
I (2598) wifi station: sent subscribe successful, msg_id=25677
I (2648) wifi station: MQTT_EVENT_SUBSCRIBED, msg_id=25677
I (314258) wifi station: MQTT_EVENT_DATA
I (314258) wifi station: TOPIC=/topic/test
I (314258) wifi station: DATA=hello world
```

9.3 Ensuring MQTT Data Security

Data transmitted using the MQTT protocol is in plain text, which means it can be intercepted if not encrypted. In Chapter 8.4.1, the TLS protocol is introduced as a means to ensure that data can only be decrypted by the communicating parties, thereby safeguarding data security and legitimacy.

Similarly, TLS can also be used for encryption in cloud communication over MQTT. Since it has already been covered in Chapter 8.4.1, this section only introduces what the certificates in the TLS handshake mean and what functions they perform, how to generate certificates locally, and how to set up a mutual authentication TLS environment based on the local MQTT broker.

9.3.1 Meaning and Function of Certificates

1. Introduction

Certificates, also known as public-key certificates (PKC), contain personal information such as user name, organization, email, the user's public key, and the digital signature of a certification authority or certifying authority (CA). You can think of a certificate as a personal identity card, with the public key serving as the card number, identifying which individual it represents. A CA is like a police station that issues the card. It can be an international organization, government entity, for-profit company, or general individual.

2. Certificate generation

(1) User A generates a private-public key pair locally using an asymmetric encryption algorithm.

(2) User A submits the locally generated public key and certificate information file to a CA for digital signing and certificate generation.

(3) The CA generates a private-public key pair locally, which is the `ca.key` mentioned in a later example. The CA uses its private key to digitally sign User A's public key and issues the certificate.

(4) User B obtains the CA's public key, which is publicly available, and uses it to verify the legitimacy of the digital signature in User A's certificate. If the verification is successful, it is confirmed that the public key in User A's certificate belongs to User A.

(5) To send data to User A, User B only needs to encrypt the data using the public key in User A's certificate and send it. User A then decrypts the data with its own private key.

So far, we have covered the generation of User A's certificate and the data communication process between User A and User B. However, it only involves one-way authentication of TLS certificate—User A is like the server side, User B is like the client side, and this process can be considered as the client's verification of the server's certificate. A similar process is followed for the server's verification of the client's certificate.

3. Certificate function

As indicated by the above certificate generation process, certificates are a means to verify the legitimacy of the peer device. Only when it is legitimate can the transmitted data can be secured without the risk of being leaked.

4. Certificate standard

Certificates adopt the common format X.509. All certificates comply with the ITU-T X.509 international standard. The structure of X.509 certificates is described and encoded using Abstract Syntax Notation One (ASN1).

A certificate typically consists of the following fields:

- **Version Number**: the version number of the specification. The current version is 3, corresponding to the value 0x2.
- **Serial Number**: the unique serial number maintained by the CA and assigned to each certificate for tracking and revocation. The maximum size is 20 bytes.
- **Signature Algorithm**: the algorithm used for digital signatures.
- **Validity**: the validity period of the certificate, including start and end dates.
- **Subject**: the identifier information of the certificate holder, namely the personal information mentioned above.
- **Subject Public Key Info**: protected information related to the public key, including the public key algorithm and subject unique identifier.

5. Certificate format

Privacy Enhanced Mail (PEM) is a common format for X.509 certificates. PEM files are usually seen with the extensions `.crt` or `.cer` (for certificates), `.key` (for private keys), and `.csr` (for certificate signing request).

The PEM file is a text file that usually contains headers, footers, and the content blocks encoded in Base64.

9.3.2 Generating Certificates Locally

OpenSSL is an open-source Secure Socket Layer (SSL) cryptographic library that provides functions for algorithms, key and certificate encapsulation management, and SSL protocol implementation. It consists of three parts: an SSL protocol library, command-line tools for applications, and cryptographic algorithm libraries. The following examples demonstrate how use it to generate certificates and keys on Linux.

1. Generating private keys for the certificate

This command generates a private key (2048 bits) for the certificate. The public key can be extracted from it.

```
$ openssl genrsa -out ca.key 2048
Generating RSA private key, 2048 bit long modulus
.....................................................................
.........................................+++
.......+++
e is 65537 (0x10001)
```

This command generates a private key (2048 bits) for the server certificate.

```
$ openssl genrsa -out server.key 2048
Generating RSA private key, 2048 bit long modulus
..............+++
.....................+++
e is 65537 (0x10001)
```

This command generates a private key (2048 bits) for the client certificate.

```
$ openssl genrsa -out client.key 2048
Generating RSA private key, 2048 bit long modulus
.............................................+++
.....................................................+++
e is 65537 (0x10001)
```

The recommended minimum key length for RSA algorithm is 2048 bits. If the key length is 1024 bits, `mbedtls` will reject TLS negotiation due to low security.

2. Generating CSRs for the certificate

This command generates a certificate sign request (CSR) that is required by the CA certificate. Enter the required information as prompted. The `Organization Name` can be entered as desired because this is only for local use.

```
$ openssl req -out ca.csr -key ca.key -new
You are about to be asked to enter information that will be incorporated into
your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
```

```
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:CN
State or Province Name (full name) [Some-State]:
Locality Name (eg, city) []:
Organization Name (eg, company) [Internet Widgits Pty Ltd]:IOT Certificate Test
Organizational Unit Name (eg, section) []:
Common Name (e.g.  server FQDN or YOUR name) []:
Email Address []:

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:
An optional company name []:
```

This command generates a CSR that is required by the server certificate. Note that the `Common Name` field should be filled with the domain name or IP address of the server.

```
$ openssl req -out server.csr -key server.key -new
You are about to be asked to enter information that will be incorporated into
your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:CN
State or Province Name (full name) [Some-State]:
Locality Name (eg, city) []:
Organization Name (eg, company) [Internet Widgits Pty Ltd]:MQTT Server
Organizational Unit Name (eg, section) []:
Common Name (e.g.  server FQDN or YOUR name) []:192.168.3.4
Email Address []:

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:
An optional company name []:
```

3. Generating CA certificate, server certificate, and client certificate

This command generates the CA certificate `ca.crt`.

```
$ openssl x509 -req -in ca.csr -out ca.crt -sha256 -days 5000 -signkey ca.key
Signature ok
subject=/C=CN/ST=Some-State/O=IOT Certificate Test
Getting Private key
```

This command generates the server certificate `server.crt`.

```
$ openssl x509 -req -in server.csr -out server.crt -sha256 -CAcreateserial
-days 5000 -CA ca.crt -CAkey ca.key
Signature ok
subject=/C=CN/ST=Some-State/O=MQTT Server/CN=192.168.3.4
Getting CA Private Key
```

This command generates the client certificate `client.crt`.

```
$ openssl x509 -req -in client.csr -out client.crt -sha256 -CAcreateserial
-days 5000 -CA ca.crt -CAkey ca.key
Signature ok
subject=/C=CN/ST=Some-State/O=MQTT Client/CN=192.168.3.5
Getting CA Private Key
```

NOTE

Do not use the SHA1 algorithm, as `mbedtls` may reject TLS negotiation due to low security.

9.3.3 Configuring MQTT Broker

Section 9.2.5 described how to set up an MQTT broker on Windows or Linux using Mosquitto, and this section will introduce how to do it over TLS.

Firstly, in the root directory of `mosquitto`, open the configuration file `mosquitto.conf`, and add the absolute path to the files that are generated in Section 9.3.2, including the CA certificate (`ca.crt`), the server certificate (`server.crt`), and the private key of the server certificate (`server.key`). The command is as follows:

```
$ port 8883
certfile {absolute path}/server.crt
keyfile {absolute path}/server.key
cafile {absolute path}/ca.crt
require_certificate true
use_identity_as_username true
```

Then, restart Mosquitto and load the configuration file.

```
$ mosquitto -c mosquitto.conf -v
1635927859: mosquitto version 1.6.3 starting
1635927859: Config loaded from mosquitto.conf.
1635927859: Opening ipv4 listen socket on port 8883.
1635927859: Opening ipv6 listen socket on port 8883.
```

9.3.4 Configuring MQTT Client

In Section 9.2.5, we have covered how to set up an MQTT client based on ESP-IDF using MQTT over TCP, but this approach cannot guarantee data security. So, in this section, we will introduce a more secure approach, which is setting up the client using MQTT over TLS.

```
1.extern const uint8_t client_cert_pem_start[] asm("_binary_client_crt_start");
2.  extern const uint8_t client_cert_pem_end[] asm("_binary_client_crt_end");
3.  extern const uint8_t client_key_pem_start[] asm("_binary_client_key_start");
4.  extern const uint8_t client_key_pem_end[] asm("_binary_client_key_end");
5.  extern const uint8_t server_cert_pem_start[] asm("_binary_ca_crt_start");
6.  extern const uint8_t server_cert_pem_end[] asm("_binary_ca_crt_end");
7.
8.  #define CONFIG_BROKER_URL "mqtts://192.168.3.4/"
9.
10. //Configure MQTT URI
11. esp_mqtt_client_config_t mqtt_cfg = {
12.     .uri = CONFIG_BROKER_URL,
13.     .client_cert_pem = (const char *)client_cert_pem_start,
14.     .client_key_pem = (const char *)client_key_pem_start,
15.     .cert_pem = (const char *)server_cert_pem_start,
16. };
```

(1) Load the client certificate (`client.crt`), the client private key (`client.key`), and the CA certificate that authenticates the server (`ca.crt`).
(2) Change the previous MQTT connection to `mqtts`.
(3) Use the default port number `8883`.
(4) Modify the `CMakeLists.txt` file to load the certificates into the firmware during compilation. The code is as follows:

```
1.  target_add_binary_data(${CMAKE_PROJECT_NAME}.elf "main/client.crt" TEXT)
2.  target_add_binary_data(${CMAKE_PROJECT_NAME}.elf "main/client.key" TEXT)
3.  target_add_binary_data(${CMAKE_PROJECT_NAME}.elf "main/ca.crt" TEXT)
```

After compilation and flashing, connect the device to Wi-Fi. Then, both the client and server will show a successful connection log, and the client will subscribe to the topic `/topic/test`. The server's log is as follows:

```
1635927859: mosquitto version 1.6.3 starting
1635927859: Config loaded from mosquitto.conf.
1635927859: Opening ipv4 listen socket on port 8883.
1635927859: Opening ipv6 listen socket on port 8883.
1635927867: New connection from 192.168.3.5 on port 8883.
1635927869: New client connected from 192.168.3.5 as ESP32_2465F1 (p2, c1, k120,
u'192.168.3.5').
1635927869: No will message specified.
1635927869: Sending CONNACK to ESP32_2465F1 (0, 0)
1635927869: Received SUBSCRIBE from ESP32_2465F1
1635927869:     /topic/test (QoS 0)
1635927869: ESP32_2465F1 0 /topic/test
1635927869: Sending SUBACK to ESP32_2465F1
```

Now, use `mosquitto_pub` to send "hello world" to the topic `/topic/test`. Check if the device can receive it. The command is as follows:

```
$ mosquitto_pub -h 192.168.3.4 -p 8883 -t \/topic/test" -m 'hello world'
--cafile ca.crt --cert client.crt --key client.key
```

Here is the log on the device side:

```
I (1600) esp_netif_handlers: sta ip: 192.168.3.5, mask: 255.255.255.0, gw: 192.168.3.1
I (1600) wifi station: got ip:192.168.3.5
I (1600) wifi station: connected to ap SSID:myssid password:12345678
I (1610) wifi station: Other event id:7
W (1630) wifi:<ba-add>idx:0 (ifx:0, 34:29:12:43:c5:40), tid:0, ssn:4, winSize:64
I (4110) wifi station: MQTT_EVENT_CONNECTED
I (4120) wifi station: sent subscribe successful, msg_id=42634
I (4140) wifi station: MQTT_EVENT_SUBSCRIBED, msg_id=42634
I (10290) wifi station: MQTT_EVENT_DATA
I (10290) wifi station: TOPIC=/topic/test
I (10290) wifi station: DATA=hello world
```

9.4 Practice: Remote Control through ESP RainMaker

After reading the previous chapters, you should have a basic understanding of Wi-Fi configuration and MQTT protocol. In this section, we will delve deeper into the Smart Light project discussed in this book. We will utilise the ESP RainMaker to provide the smart light with more functions, including remote control, local control, OTA upgrade, and scheduling. Besides, we will enable third-party applications Alexa and Google Home to control the smart light using Skill and facilitate voice control using voice assistants such as Alexa and Google Assistant.

A standard voice assistant allows you to switch a smart light on or off, as well as adjust its brightness. If the smart light supports colour and colour temperature adjustments, you can send specific voice commands to alter these settings. Moreover, you can use voice assistant-enabled speakers such as Echo and Nest to discover and control the light.

9.4.1 ESP RainMaker Basics

Before delving into the functions of ESP RainMaker, this section first explains some fundamental concepts that will be mentioned in the description of the ESP RainMaker framework (backend and frontend). The ESP RainMaker framework is illustrated in Figure 9.6.

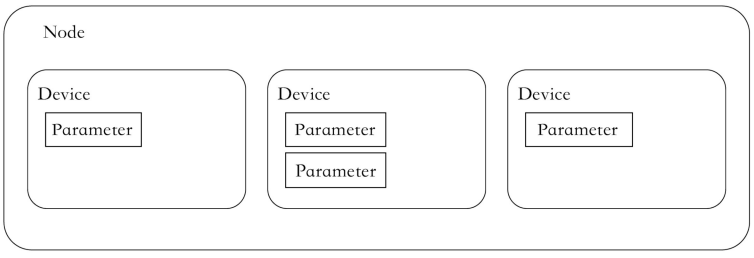


Figure 9.6. ESP RainMaker framework

Node

It refers to the device model that represents the physical device (such as ESP32-C3) in the cloud. Each node has a unique identifier, namely, node ID. It is the smallest operational unit and a representation of the physical device in the ESP RainMaker framework.

Node attribute

It is used to better describe and define the functions of nodes. ESP RainMaker has set default metadata for the node, including `fw_version` and `model`. The name and type that are set when a node is created also belong to the default metadata. You can also add your own information to the metadata to better describe the node.

Device

It is a logical entity that the user can control, such as a switch, smart light, temperature sensor, or fan. Unlike a node, a device is the smallest unit that can be operated at the user level.

Device attribute

Similar to node attribute, it is used to better describe and define functions of devices.

Service

In the ESP RainMaker framework, a service is a very similar entity to a device. The main difference is that the service does not require operations from the user. For example, the OTA upgrade service has some states that do not require any operation and management from the user.

Parameter

It is used to implement functions of devices and services, such as the power status, brightness, and colour of a smart light, and status updates during OTA upgrades.

The concepts of nodes, devices, parameters, and services in the ESP RainMaker framework can aptly describe the form and functions of the product. For example, to create a smart light with controllable power status, brightness, colour, and scheduled switching, the light is represented by a node and a device, the power status, brightness, and colour are controlled by parameters, and the scheduling function is achieved by a service.

9.4.2 Node and Cloud Backend Communication Protocol

The communication between the node and cloud backend is encrypted using TLS over MQTT, and their identities are mutually authenticated using X.509 certificates. The private key used for the node connection is automatically generated on the node.

During the first Wi-Fi provisioning, ESP32-C3 obtains a certificate through Assisted Claiming, and saves it in its flash. The process for ESP32-C3 to use Assisted Claiming is shown in Figure 9.7.

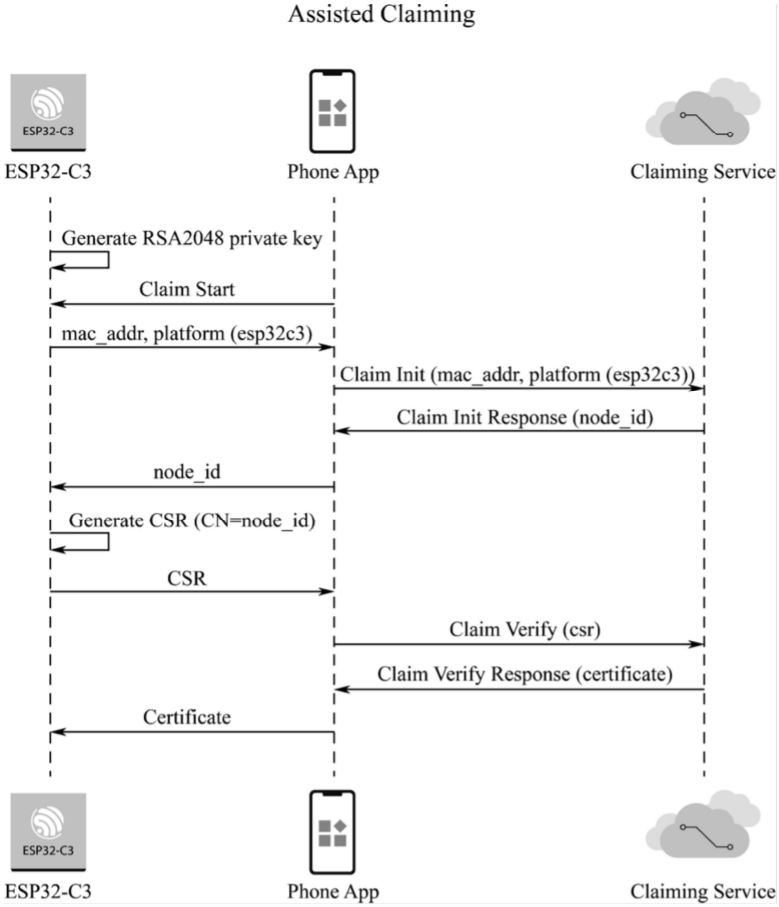


Figure 9.7. ESP32-C3 Assisted Claiming

(1) ESP32-C3 generates an RSA2048 private key, uses its MAC address as the initial node ID, and then sends relevant messages to the smartphone app.

(2) During the first provisioning, the app and the Claiming Service authenticate each other's identity. Once the authentication is successful, the receiving server issues a node ID, which is then forwarded by the app to ESP32-C3.

(3) ESP32-C3 generates a CSR with the CN field set as the node ID. Then, the CSR is forwarded by the app to the Claiming Service.

(4) The Claiming Service verifies the CSR and issues the certificate, which is then forwarded by the app to ESP32-C3.

The node ID serves not only as a means of identifying a node during certificate application but also as a way to map to a user and filter MQTT messages. For example, a node can only subscribe to topics with a specific prefix (`node/<node_id>/*`) and publish messages to those topics.

ESP RainMaker defines some default messages, including configuration messages, control messages, status messages, initial status messages, mapping messages, OTA upgrade messages, and warning messages. These messages are packaged in JSON and sent to the cloud backend over MQTT.

The configuration message is published by nodes through `node/<node_id>/config`. It contains information about the node itself, its attributes, devices, device attributes, services, and parameters. Here is an example.

```
1.  //Configuration message for led_light
2.  {
3.      "node_id": "xxxxxxxxxx",                    //Node ID
4.      "config_version": "2020-03-20",             //Configuration version
5.      "info": {                                   //Node information
6.          "name": "ESP RainMaker Device",
7.          "fw_version": "1.0",
8.          "type": "Lightbulb",
9.          "model": "led_light"
10.     },
11.     "devices": [                                //Devices of this node
12.         {
13.             "name": "Light",
14.             "type": "esp.device.lightbulb",
15.             "primary": "Power",
16.             "params": [                         //Device parameters
17.             {
18.                 "name": "Name",
19.                 "type": "esp.param.name",
20.                 "data_type": "string",
21.                 "properties": ["read", "write"]
22.             },
23.             {
24.                 "name": "Power",
25.                 "type": "esp.param.power",
26.                 "data_type": "bool",
27.                 "properties": ["read", "write"],
28.                 "ui_type": "esp.ui.toggle"
29.             },
30.             ......
31.             ]
32.         }
33.         ],
34.         "services": [                       //Node services
35.         {
36.             "name": "OTA",
37.             "type": "esp.service.ota",
38.             "params": [
```

```
            {
                "name": "Status",
                "type": "esp.param.ota_status",
                "data_type": "string",
                "properties": ["read"]
            }
            ......
            ]
        }
    ]
}
```

The smartphone app can obtain the unique identifier of the product by parsing `node_id`, the device information and number by parsing `devices`, the services by parsing the `services`, and the read and write permissions of the app by parsing `properties`. If `ui_type` of `params` in `devices` is set, the app will display the corresponding UI. For more information on the use of standard parameters, standard devices, and standard UI, please refer to Section 9.4.7.

The downlink control message is used the app and third-party applications to control nodes. It contains device parameters that need to be updated. To receive such messages, a node needs to subscribe to the topic `node/<node_id>/remote`. Here is an example of control messages.

```
{
    "Light": {
        "Power": false
    }
}
```

A node can actively report its status messages through the topic `node/<node_id>/params/local`. The cloud backend will cache the parameters in the message and push them to the clients that have enabled the push function.

The mapping message is used to map nodes to users. An unmapped node needs to be mapped to a user first to ensure that only that user has access to it. The mapping request occurs during the Wi-Fi configuration phase. During Wi-Fi provisioning, the device receives the user ID and security key from the smartphone. Once the node is connected to the cloud backend, it will concatenate the user ID and security key with its own node ID and send them to the cloud backend. Here is an example of mapping messages.

```
{
    "node_id": "112233AABBCC",
    "user_id": "02e95749-8d9d-4b8e-972c-43325ad27c63",
    "secret_key": "9140ef1d-72be-48d5-a6a1-455a27d77dee"
}
```

After receiving the above message, the cloud backend will check whether it receives the

same security key from the app. If yes, it will map the user to the device. The mapping process is shown in Figure 9.8.

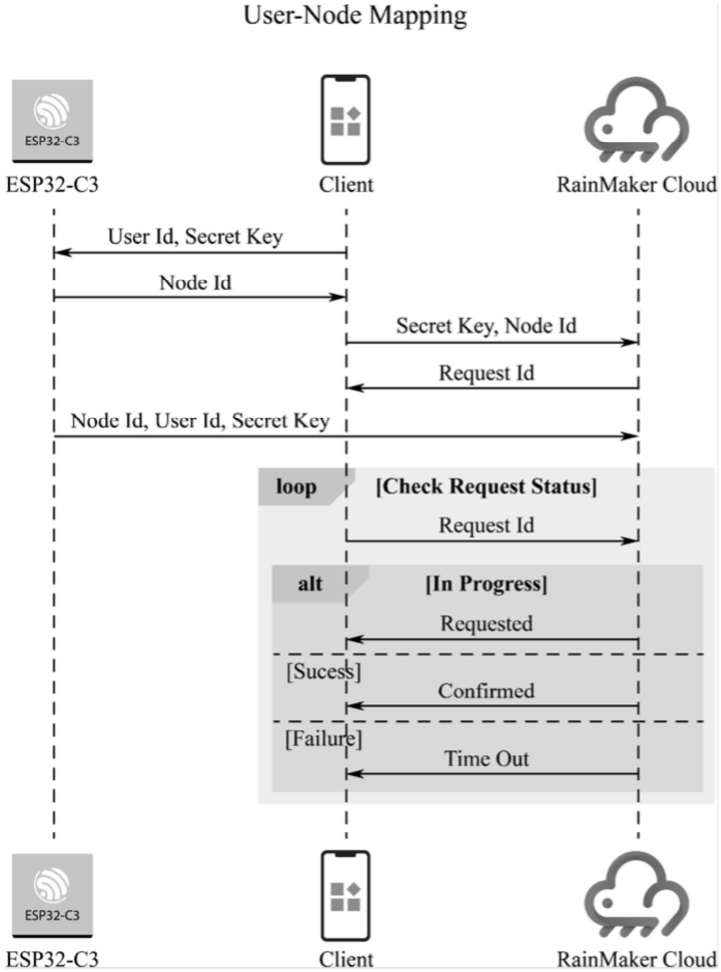


Figure 9.8. Mapping process

The OTA upgrade message is used to implement OTA upgrades for nodes. It uses three MQTT topics: `node/<node_id>/otafetch`, `node/<node_id>/status`, and `node/<node_id>/otaurl`. These topics respectively report OTA upgrade status, distribute OTA upgrade firmware, and query OTA upgrade tasks. The code is as follows:

```
1.  //Distribute OTA upgrade firmware
2.  {
3.      "url": "<ota_image_url>",
4.      "ota_job_id": "<ota_job_id>",
5.      "file_size": "<num_bytes>"
6.  }
7.
8.  //Query OTA upgrade tasks
9.  {
```

```
10.     "node_id": "<node_id>",
11.     "fw_version": "<fw_version>"
12. }
13.
14. //Report OTA upgrade status
15. {
16.     "ota_job_id": "<ota_job_id>",
17.     "status": "<in-progress/success/fail>",
18.     "additional_info": "<additional_info>"
19. }
```

The warning message is a type of push messages used to notify and remind users. A node can publish warning messages via the topic `node/<node_id>/alert`. After receiving a warning message, the app pushes it to the smartphone notification bar. All the data in the cloud backend have push properties, and the use of this topic explicitly marks the data as a notification that needs to be actively pushed to the smartphone's notification bar. Here is an example of warning messages.

```
1.
2.  "esp.alert.str":  "alert"
3.
```

9.4.3 Communication between Client and Cloud Backend

ESP RainMaker offers two client tools: app and CLI, both of which are implemented using the RESTful API. This section briefly explains how to use the CLI tool that comes with the device SDK to communicate with the cloud backend.

The CLI tool is a Python-based submodule of the esp-rainmaker repository, under the `esp-rainmaker/cli` directory. To use it, please refer to Chapter 4 to set up the ESP-IDF environment and export the ESP-IDF environment variables. You can verify whether the ESP-IDF and Python environments are ready by running the following commands:

```
# Print ESP-IDF version
$ idf.py --version
ESP-IDF v4.3.2

# Print Python version
$ python3 --version
Python 3.6.9
```

A similar Shell output to the above indicates that the ESP-IDF environment is ready. Note that the CLI tool depends on Python 3.x, and the older versions need upgrading.

After the ESP-IDF environment is ready, use pip to install the Python dependencies of the CLI tool with the following commands:

```
$ cd your RainMaker path/esp-rainmaker/cli
$ pip install -r requirements.txt
```

```
Collecting argparse
  Using cached argparse-1.4.0-py2.py3-none-any.whl (23 kB)
...
...
...
Installing collected packages: cryptography, argparse
  Attempting uninstall: cryptography
    Found existing installation: cryptography 2.9.2
    Uninstalling cryptography-2.9.2:
      Successfully uninstalled cryptography-2.9.2
Successfully installed argparse-1.4.0 cryptography-2.4.2
WARNING: You are using pip version 21.1.2; however, version 21.3.1 is available.
```

Once the environment is set up, you can use the CLI tool to communicate with the cloud backend. All the commands supported by the CLI are listed in Table 9.4, and you can view the usage of each command by running `python3 rainmaker.py {h`. Additionally, you can use the parameter `-h` together with each command to view more help information.

Table 9.4. CLI commands

Command	Description
`signup`	Sign up for ESP RainMaker
`login`	Login to ESP RainMaker
`logout`	Logout current (logged-in) user
`forgotpassword`	Reset the password
`getnodes`	List all nodes associated with the user
`getnodeconfig`	Get node configuration
`getnodestatus`	Get online/offline status of the node
`setparams`	Set node parameters
`getparams`	Get the last parameter of the node in the cloud
`removenode`	Remove user node mapping
`provision`	Provision the node to join Wi-Fi network
`getmqtthost`	Get the address of the MQTT host that the node connects to
`claim`	Perform host driven claimming to the node and get the MQTT cerficate
`test`	Test whether the node has been mapped to the user
`otaupgrade`	Distribute OTA upgrade information
`getuserinfo`	Get detailed information of the logged-in user
`sharing`	Share the node

The claim command in the CLI tool is for host driven claiming, which is no longer supported in ESP32-C3. Instead, the more convenient Self Claiming is supported in ESP32-C3.

Before using any other command, you need to first run the `signup` command to sign up for an ESP RainMaker account:

```
$ cd your RainMaker path/esp-rainmaker/cli
$ cd python3 rainmaker.py signup someone@example.com
Choose a password
Password :
Confirm Password :
Enter verification code sent on your Email.
Verification Code : 973854
Signup Successful
Please login to continue with ESP Rainmaker CLI
```

Check the verification code in your email, as shown in Figure 9.9.

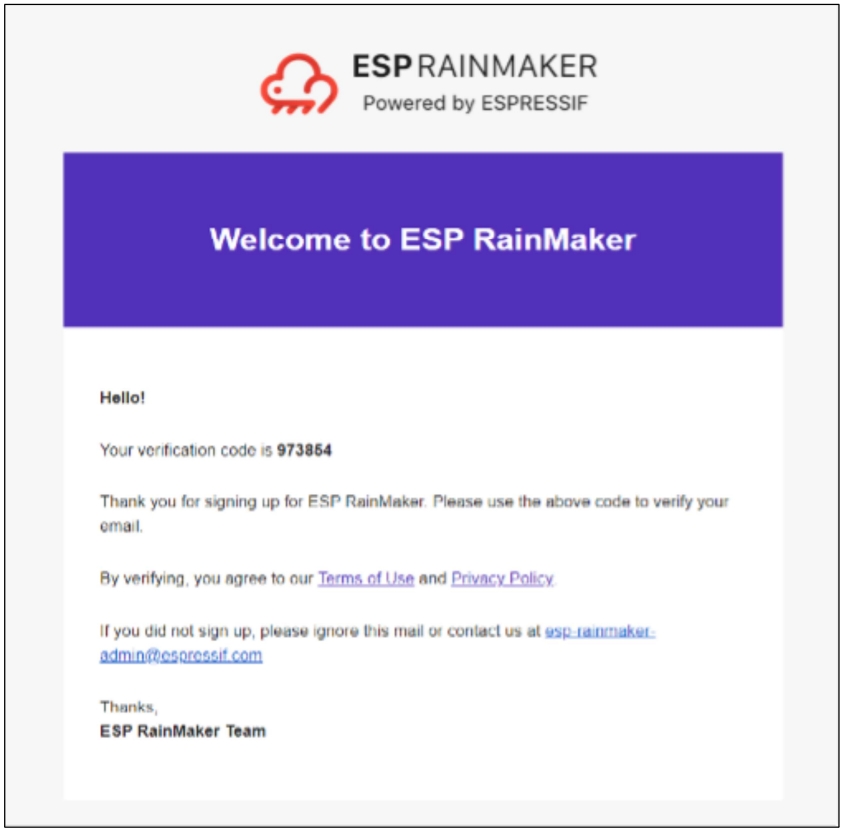


Figure 9.9. Verification code in email

Then, log in.

Execute the `login` command, and the Shell will open a web page, as shown in Figure 9.10. Enter your account and password in the box.

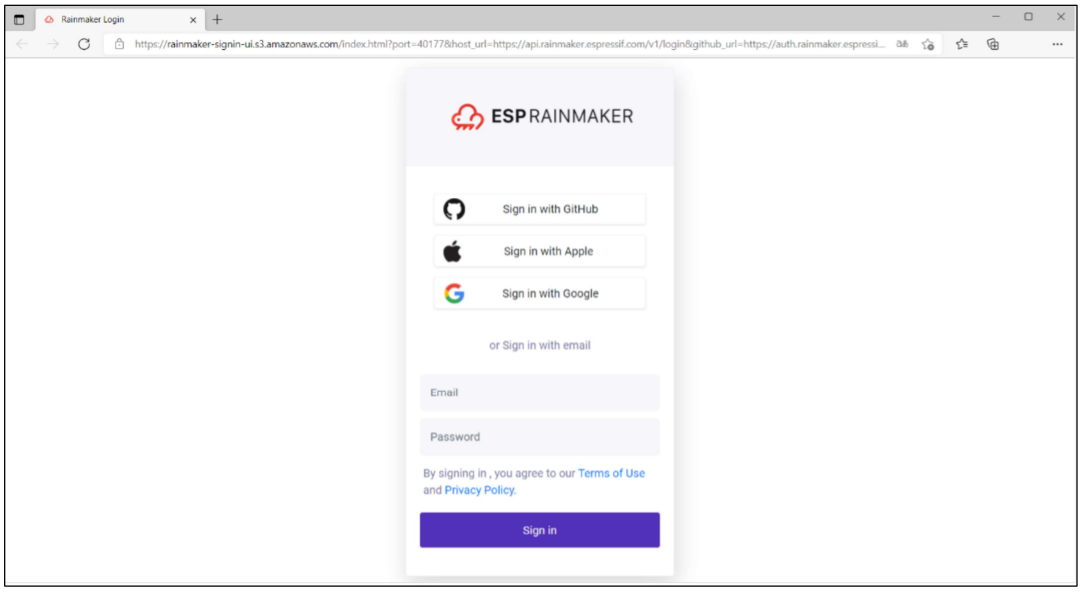


Figure 9.10. Log in to ESP RainMaker using web browser

Alternatively, you can enter the command `login` together with the parameter `-email` to directly log in using CLI:

```
$ python3 rainmaker.py login --email someone@example.com
Password:
Login Successful
```

9.4.4 User Roles

As mentioned earlier, user-node mapping aims to ensure that each node is controlled by a unique user. There are two types of users in ESP RainMaker: admin users and end users.

Admin user:

a user who owns the MQTT certificate of a given node or gets the certificate through claiming service. Admin users can access nodes via the ESP RainMaker dashboard, push OTA firmware updates, and use ESP Insight for remote monitoring, but cannot read or write node parameters.

End user:

a user who has the control permissions to a given node. End users can set and obtain node parameters and configuration but cannot view nodes via the ESP RainMaker dashboard. They are subdivided into primary users and secondary users.

Primary user:

a user who last performs the user-node mapping. Primary users can access node configuration, read/write node parameters, and add/remove/view other secondary users.

Secondary user:

any user who gets access to a node via node sharing. Secondary users can access node configuration and read/write node parameters, but cannot add/remove/view other secondary users.

9.4.5 Basic Services

ESP RainMaker services are practical examples that integrate specific functions to facilitate secondary development and enrich the functionality of nodes. For example, the scheduling service provides devices with the offline timing/countdown function; the system service offers the remote reboot and factory reset functions; the time & time zone service enables the time zone switching function; the OTA upgrade service provides the remote update function; and the local control service allows fast, stable, and secure LAN communication. These services can be quickly implemented with simple configuration.

1. Time & time zone service

Fetching time is one of the most critical tasks for an IoT device after connecting to the Internet, especially when the scheduling service is enabled. In ESP RainMaker, there are two important concepts: time and time zone.

Time is typically fetched using Simple Network Time Protocol (SNTP). The ESP RainMaker SDK provides an abstraction layer over the SNTP component in ESP-IDF, making it easy for you to synchronise and check time. The code is as follows:

```
1. /*Initialise time synchronisation. This will call the SNTP component internally and
2. set the SNTP server through sntp_server_name passed by esp_rmaker_time_config_t*/
3. esp_err_t esp_rmaker_time_sync_init(esp_rmaker_time_config_t *config);
4.
5. //Check if time has been synchronised by comparison with the standard time 1546300800
6. bool esp_rmaker_time_check(void);
7.
8. //Wait for time to be synchronised
9. esp_err_t esp_rmaker_time_wait_for_sync(uint32_t ticks_to_wait);
```

As countries and regions in different longitudes have different local times and time zones, the `TZ` environment variable and the `tz_set()` function are provided by ESP-IDF to set the time zone. RainMaker provides an abstraction layer over this and provides multiple ways of setting it. For example:

(1) Using the **C API**.

```
1.  //Set time zone using the timezone region string
2.  esp_err_t esp_rmaker_time_set_timezone(const char *tz);
3.
4.  //Set time zone using the POSIX format
5.  esp_err_t esp_rmaker_time_set_timezone_posix(const char *tz_posix);
```

(2) Modifying `Default Timezone` in `menuconfig`. To use this method, you need to have